

Resource isolation: You can restrict and assign resources like CPU, memory, and disk input/output to containers using cgroups.

Security: By using user namespaces and unprivileged containers, Linux containers can be isolated and secured with the right configuration.

Flexibility: Like traditional virtual machines, LXC offers flexibility by allowing full-fledged Linux distributions to run within containers.

With Linux containers, you can run multiple isolated Linux environments on a single host and share a kernel thanks to a lightweight virtualisation technique. Using namespaces and control groups (cgroups), LXCs

make sure that every container has its own filesystem, network interfaces, and process tree. Prominent features of LXCs are resource allocation, networking, and storage configuration files, as well as container management commands like *lxc-create*, *lxc-start*, and *lxc-attach*.

Linux containers are a great choice for system-level virtualisation because of their efficiency, flexibility, and near-native performance. This is especially useful when multiple services or full Linux distributions need to run in secure, isolated environments. **END** 

References

- <https://en.wikipedia.org/wiki/LXC> [LXC Project]
- <https://www.docker.com/blog/lxc-vs-docker/> [Overview of LXC]

By: Dr Dharmendra Patel

The author is a professor at Smt Chandaben Mohanbhai Patel Institute of Computer Applications, Gujarat. His areas of interest are data mining, web mining, image processing, Internet of Things and software engineering.

■■ Continued to from page...52

This configuration allows for scalable, secure networking with eBPF-based load balancing and traffic filtering.

Observability with BPFTrace:

For deeper observability, you can deploy BPFTrace in your Kubernetes environment to capture system events and trace application behaviour.

Install BPFTrace in your nodes:

```
sudo apt-get install bpftrace
```

Write a simple trace script to monitor system call:

```
bpftrace -e 'tracepoint:syscalls:sys_
enter_execve { printf("Executing %s\n",
str(args->filename)); }'
```

The integration of eBPF with Kubernetes brings advanced networking capabilities, security, and deep observability to containerised

environments. eBPF enables fine-grained control over network traffic, improves performance through efficient packet processing, and provides real-time insights into system behaviour. By leveraging tools like Cilium for networking and BPFTrace for observability, Kubernetes administrators and developers can unlock powerful new ways to monitor, secure, and optimise their clusters.

As the cloud-native landscape continues to evolve, the role of eBPF in Kubernetes is poised to grow, making it an essential tool for modern infrastructure management. Whether you're looking to enhance performance, secure your network, or gain deeper insights into system behaviour, eBPF's integration with Kubernetes offers unparalleled benefits. **END** 

References

- <https://www.groundcover.com/ebpf/ebpf-kubernetes>
- <https://cilium.io/blog/2020/11/10/ebpf-future-of-networking/>
- <https://www.wiz.io/academy/ebpf-in-kubernetes>
- <https://blogs.cisco.com/applications/exploring-ebpf-a-new-approach-to-observing-kubernetes-workloads>
- <https://kubernetes.io/blog/2017/12/using-ebpf-in-kubernetes/>

By: Dhanashree Yevle

The author is a research scholar specialising in artificial intelligence, machine learning, and deep learning.